SmartPot

SOFTWARE LIFECYCLE

SmartPot End-to-End Lifecycle

Formulation, Management, Analysis, Design, Implementation, Testing, and Continuous Improvement

Critical evaluation across all engineering lifecycle phases: planned milestones, evidentiary artifacts, architectural discrepancy resolution, and operational lessons learned.



DOCUMENT / DOCUMENTO	VERSION / VERSIÓN	TEAM / EQUIPO	PROJECT / PROYECTO
Software Lifecycle	**1.1 · October 2026**	**SmartPotTech**	**smartpot.app**

DOCUMENT

Document Summary

FIELD	VALUE
Project	SmartPot · smartpot.app
Organization	SmartPotTech
Document	Software Lifecycle: Formulation, Management, Analysis, Design, Construction, Testing, and Continuous Improvement
Version	1.1 · October 2026
Period	September 2024 to October 2026
Sources	Project formulation artifacts (problem tree, 5 Whys, PESTEL, stakeholder matrix, logical framework), market study, project charter, schedule and critical path reports, research proposal, software requirements, use cases, UML designs, and agronomic knowledge base; verified against repository git histories and production runtime
Companion Documents	Technical Documentation: active production platform reference. Project Journey: requirement-by-requirement and diagram-by-diagram comparative evolution
How to Read	Each part corresponds to a core software engineering lifecycle discipline and contains three dimensions: original planned design, critical evaluation of discrepancies, and active production code evidence

CONTENTS · CONTENIDO

Contents / Contenido

1. Project Timeline and Inception

2. Problem Identification

- 2.1 The Concept
- 2.2 Problem Tree & Root Causes
- 2.3 SMART Objectives

3. Environment, Stakeholders, and Logical Framework

- 3.1 PESTEL Analysis Summary
- 3.2 Stakeholder Management

4. Market and Technical Feasibility

- 4.1 Market Strategy
- 4.2 Technical Study Summary

5. Scope, Schedule, and Resource Management

- 5.1 Work Breakdown Structure (WBS)

5.2 Critical Path Method (CPM) and Resource Leveling
5.3 Risk Management

6. Requirements Engineering and Domain Modeling

6.1 Functional Requirements (RF-001 to RF-017)
6.2 Non-Functional Requirements (RNF-001 to RNF-006)

7. Software Architecture and Design Patterns

7.1 Architecture Evolution: From Stack A to Stack B
7.2 Core Design Patterns Applied

8. Construction and Implementation Engineering

8.1 13-Repository Modular Ecosystem
8.2 AI Reasoning and Continuous Learning

9. Quality Assurance and Verification

9.1 Multi-Layer Verification

10. Continuous Improvement and Engineering Kaizen

10.1 Key Performance Indicators (KPIs)
10.2 Future Backlog
10.3 Lessons Learned

PART I

IT Project Formulation and Evaluation

SECTION 01

Project Timeline and Inception

In Plain Words

SmartPot evolved from a solitary desktop potted plant equipped with an ESP32 publishing sensor data to a Telegram chat, into a production-grade distributed IoT and AI platform serving urban hydroponic micro-farming. Across its lifecycle, the project traversed academic software engineering courses, formal IT project formulation, market research, applied AI research, and comprehensive architectural refactoring to achieve its current production state at smartpot.app.

SECTION 02

Problem Identification

2.1 The Concept

Urban hydroponics offers significant yield advantages and up to 70% water savings compared to traditional soil farming. However, without automated closed-loop control, crop survival depends entirely on manual oversight. Manual monitoring leads to delayed pH adjustments, undetected nutrient imbalances, irregular watering, and catastrophic plant loss.

2.2 Problem Tree & Root Causes

- **Direct Causes:**
- Inadequate sensorization and lack of automated dosing actuators.
- Sporadic, subjective manual measurements prone to human error.
- Extreme ambient environmental volatility (temperature, solar radiation, humidity).
- **Core Problem:** Absence of an affordable, automated, and intelligent closed-loop monitoring and control system for urban hydroponics.
- **Direct Effects:** High crop mortality, wasted water and fertilizer, operational frustration, and restricted adoption of vertical agriculture.

2.3 SMART Objectives

- **Specific:** Develop an automated IoT platform integrating environmental sensors, automated actuators, time-series telemetry persistence, and edge AI reasoning.
- **Measurable:** Maintain pH (5.5–6.8) and TDS (560–2450 ppm) within ideal species bands 95% of operational time; reduce manual inspection labor by 80%.
- **Achievable:** Utilize low-cost commodity microcontrollers (ESP32) and open-source backend runtimes (Go, Rust, TimescaleDB).
- **Relevant:** Directly advance UN Sustainable Development Goals 2, 9, 11, and 12.
- **Time-Bound:** Deliver a fully tested and certified production platform within the academic and development timeline.

SECTION 03

Environment, Stakeholders, and Logical Framework

3.1 PESTEL Analysis Summary

- **Political:** Government incentives for sustainable agriculture and urban food security.
- **Economic:** Rising urban food costs offset by hyper-local hydroponic production; accessible capital expenditure.
- **Social:** Growing consumer preference for organic, pesticide-free, locally grown produce.
- **Technological:** Availability of ultra-low-cost IoT hardware (ESP32), robust MQTT brokers, and lightweight machine learning engines.
- **Environmental:** Critical need for climate-resilient water conservation in metropolitan areas.
- **Legal:** Compliance with food safety standards, open-source licensing (MIT), and data privacy (GDPR/Habeas Data).

3.2 Stakeholder Management

Key stakeholders—including faculty evaluators, agritech researchers, open-source developers, and domestic growers—were categorized across the Power/Interest grid. Close collaboration ensured that academic rigor aligned seamlessly with industrial-grade software engineering.

SECTION 04

Market and Technical Feasibility

4.1 Market Strategy

A freemium open-source distribution model was adopted:

- The core platform, firmware, and Dockerized deployment stacks remain 100% open-source under the MIT license.
- Educational institutions, research centers, and commercial greenhouses benefit from reproducible container orchestration and turnkey documentation.

4.2 Technical Study Summary

Comparative evaluation selected the ESP32 microcontroller due to dual-core processing, hardware Wi-Fi/Bluetooth, and deep sleep efficiency. Firmware was implemented in MicroPython and Rust Embassy, communicating over MQTT with TLS to an EMQX broker.

PART II

IT Project Management

SECTION 05

Scope, Schedule, and Resource Management

5.1 Work Breakdown Structure (WBS)

The project scope was partitioned into four hierarchical levels:

1. **Formulation & Architecture:** Requirements engineering, agronomic limits, and interface contracts.
2. **Infrastructure & Persistence:** PostgreSQL/TimescaleDB schemas, Valkey cache, and EMQX broker configuration.
3. **Core Services & Firmware:** Go REST/MQTT API, ESP32 MicroPython firmware, and Rust AI microservice.
4. **Interface & Quality Assurance:** SolidJS PWA, automated E2E test suites, and AppSec hardening.

5.2 Critical Path Method (CPM) and Resource Leveling

Schedule analysis revealed that the critical path ran continuously from initial contract definition through the Go API implementation to end-to-end integration testing. Resource leveling eliminated over-allocation, ensuring consistent progress without developer burnout.

5.3 Risk Management

Key technical risks—including IoT sensor noise, MQTT network drops, BOLA vulnerabilities, and concurrency race conditions—were systematically mitigated through hardware Kalman filtering, QoS 1 delivery, strict database ownership checks, and advisory transactional locks.

PART III

Analysis

SECTION 06

Requirements Engineering and Domain Modeling

6.1 Functional Requirements (RF-001 to RF-017)

All 17 functional requirements were successfully implemented and verified in the production Stack B:

- Access management, multi-sensor calibration, real-time telemetry rendering, closed-loop irrigation, Telegram alerts, historical timeseries queries, and Sugeno fuzzy health scoring.

6.2 Non-Functional Requirements (RNF-001 to RNF-006)

- Sub-second telemetry ingestion and query response times.
- Mobile-first responsive Progressive Web App (PWA) with offline capabilities.
- Nature-aligned visual identity meeting WCAG 2.1 AA accessibility guidelines.
- Zero-trust container security with rootless execution and zero base CVEs.

PART IV

Design

SECTION 07

Software Architecture and Design Patterns

7.1 Architecture Evolution: From Stack A to Stack B

The project underwent a decisive architectural migration:

- **Stack A (Legacy Baseline):** Java 21 / Spring Boot 4, MongoDB, Mosquitto, and React. Suffered from 47 SCA vulnerabilities, heavy JVM memory overhead (1.2 GB+), and unindexed timeseries queries.
- **Stack B (Canonical Production):** Go 1.27.3 (Echo), PostgreSQL 17 + TimescaleDB, EMQX, Valkey, Rust AI microservice, and SolidJS. Achieved 100% critical vulnerability eradication, sub-50ms query latency, and an 85% memory footprint reduction.

7.2 Core Design Patterns Applied

- **Hexagonal Architecture:** Strict decoupling of domain models from external infrastructure adapters.
- **Circuit Breaker:** Resilient gRPC/REST invocation between the Go API and the Rust AI engine.
- **Singleflight:** Query coalescing preventing thundering-herd spikes on historical timeseries aggregations.
- **Advisory Transactional Locking:** `pg_advisory_xact_lock` guaranteeing serialized quota checks and eliminating TOCTOU race conditions.

PART V

Construction

SECTION 08

Construction and Implementation Engineering

8.1 13-Repository Modular Ecosystem

The platform is organized across 13 dedicated GitHub repositories:

- `SmartPot-API` (Go backend)
- `SmartPot-DB` (TimescaleDB schemas & DRP)
- `SmartPot-Cache` (Valkey)
- `SmartPot-Broker` (EMQX)
- `SmartPot-Web` (SolidJS PWA)
- `SmartPot-AI` (Rust inference)
- `SmartPot-Trainer` (Python offline learning)
- `SmartPot-DataGenerator` (Rust simulator)
- `SmartPot-IoT` (MicroPython firmware)
- `SmartPot-IoT-Python` (Desktop test harness)
- `SmartPot-MCP` (Model Context Protocol server)
- `SmartPot-Mail` (SMTP testing)
- `.github` (Central orchestration, K8s, and documentation)

8.2 AI Reasoning and Continuous Learning

The intelligence pipeline combines deterministic 21-rule heuristic validation, multi-variable Sugeno fuzzy logic, Theil-Sen trend slope forecasting, and offline machine learning model training signed with Ed25519 cryptographic keys.

PART VI

Testing

SECTION 09

Quality Assurance and Verification

9.1 Multi-Layer Verification

- **Unit & Integration Tests:** 100% pass rate across Go, Rust, Python, TypeScript, and MicroPython test suites.
- **Automated E2E Harness:** 46 multi-service operational scenarios executing on every continuous integration run.
- **Forensic Compliance Audit:** 43 rigorous compliance controls verified with zero trust, certifying zero dead code, zero false positives, and full functional wiring.

PART VII

Continuous Improvement

SECTION 10

Continuous Improvement and Engineering Kaizen

10.1 Key Performance Indicators (KPIs)

- **Availability:** 99.9% uptime across core telemetry ingestion and API services.
- **Telemetry Latency:** End-to-end ingestion latency under 250 milliseconds.
- **Crop Health Stability:** Less than 2% out-of-band operational excursions under automated control.
- **Security Posture:** 0 Critical, 0 High vulnerabilities across all production runtime containers.

10.2 Future Backlog

- Edge-quantized neural models executing directly on ESP32 microcontrollers.
- Two-Factor Authentication (TOTP 2FA) across web client sessions.
- Multi-channel physical peristaltic nutrient dispensing hardware.
- Long-range LoRaWAN telemetry support for decentralized peri-urban greenhouses.

10.3 Lessons Learned

5. **Design Clean Contracts Early:** Establishing strict MQTT v1 and OpenAPI contracts before programming prevented integration bottlenecks.
6. **Lean Containers Beat Monoliths:** Moving to Go, Rust, and `scratch` containers eliminated container maintenance overhead and slashed attack surface.
7. **Zero-Trust Verification is Essential:** Automated unit and E2E testing must verify actual hot-path execution rather than relying on commit messages or superficial mocks.

